Using DCB for Correctness — What We Found

What we found about DCB and correctness

DomainSpec v2 research — 2026-06-22

A shareable write-up for readers who know Dynamic Consistency Boundaries but not DomainSpec.

Status: research synthesis (two towers + one executed toy) Dates: 2026-06-11 (correctness tower), 2026-06-15 (saga tower + toy run) Maturity: bounded toy proof, not production. Claims below are scoped to what we actually ran.

TL;DR for a DCB reader

We explored whether Dynamic Consistency Boundaries could help a spec-driven system guarantee *correctness*. The honest answer surprised us:

1. **DCB is not the proof engine and not a validator.** It does not decide whether code matches spec, or whether a saga is behaviorally correct.
2. **DCB's real job here is freshness.** It answers one question a static validator structurally cannot: *"Was this correctness claim accepted against the same relevant facts it actually read — or did one of them change underneath it?"* That is your read-set + append-condition, repurposed as a staleness guard on an **acceptance decision**.
3. **For most facts, DCB is overkill.** Anything deterministic and cheap to recompute (spec rows, code tags, generated projections) is better handled by a content hash of the dependency set, or by simply *rerunning the check at decision time*. Staleness becomes impossible by construction, with no ledger.
4. **DCB earns its keep on exactly one class of fact:** the **non-rerunnable** ones that also need **concurrency / causality** — human approvals, LLM/agent verdicts, paid CI results, external evidence receipts, and concurrent promotions. You cannot "just rerun" a human waiver from last Tuesday.
5. We built a toy saga to prove the seam, and it behaved exactly as the theory predicted: deterministic facts guarded by hashes, one human waiver guarded by DCB, and acceptance **rejected** when the waiver changed after the read marker — while the behavioral witness itself still passed.

So the bridge decision is **borrow-carefully**: adopt the DCB read-set/append- condition *only* at the acceptance boundary, *only* for facts you cannot rerun.

What DomainSpec is (the 90-second version)

You don't need DomainSpec internals to read the rest, but you need the mental model, because it's *not* an event-sourced system — which is what makes the DCB question interesting.

DomainSpec treats a feature's behavior as **multi-artifact truth** rather than a single source file. For one feature you have:

- **Specs** — declarative documents that name concepts, relationship edges, operations, rules, events, tests, mappings, and workflows.

- **Code tags** — annotations in the implementation that bind code symbols back to the spec's concepts and edges.
- **Validators / drift tools** — checkers that compare the documentation facts against the code facts and flag mismatches (missing tags, doc-only or code-only drift, broken anchors, bad source refs).
- **Generated registries & projections** — materialized "current state" views built deterministically from the source documents (think read-models, but regenerated from files, not folded from an event log).
- **Governance surface** — work-packs, readiness signals, evidence receipts, and promotion gates that record whether a feature is allowed to advance.

The key property for our purposes: **validators answer a *static* question.**

> *Is this snapshot structurally valid right now?*

They are excellent at that. What they do **not** answer is *temporal*:

> *A correctness decision was made at some point. Were the facts it relied on still current when it was accepted — or had one of them moved?*

That gap is where we pointed DCB.

The reframing: DCB as a freshness boundary, not a correctness oracle

The instinct is to model correctness itself as a consistency boundary: `AcceptCorrectnessClaim(...)`. We think that's the wrong name, and the wrong ownership.

The artifact that *owns* the behavioral claim (the validator, the property test, the bounded witness, the human reviewer) should keep owning it. DCB should own a strictly narrower decision:

> **`AcceptProjectionFreshnessClaim`** / **`AcceptSagaProofFreshnessClaim`** — accept this already-formed correctness claim *as current evidence* only if no relevant fact it read has changed since it read it.

Mechanically this is pure DCB:

- The claim records a **read set** — every spec row, code tag, validator verdict, projection snapshot, test run, waiver, and external receipt it relied on, each carrying **tags** (`feature:…`, `concept:…`, `rule:…`, `operation:…`, `artifact:…`, `owner:…`).
- The claim records a **read marker** over that set.
- Acceptance is gated by an **append condition**: *no fact matching the read tags changed after the read marker.* If one did, the claim is stale → reject.

Same shape you already use to protect a command against concurrent writes — applied to protect a *correctness acceptance* against stale evidence.

The uncomfortable finding: where DCB is the wrong tool

We deliberately ran an *alternatives* lane that held the problem fixed and tried to solve it **without** DCB. It changed the verdict. Four non-DCB options, and two of them fully solve the problem for the *largest, cheapest* class of facts:

Approach	How it works	Beats DCB when…	Loses to DCB when…
Content-hash invalidation	Hash the content of every relied-on fact; recompute at acceptance; differ ⇒ stale ⇒ reject.	Facts are deterministic; you also want change-and-revert detection; you don't want a ledger or a read-marker model.	You need per-fact granularity, an audit narrative, or concurrency/causality.
Rerun-before-use	Don't track staleness at all; make the verdict cheap + deterministic and just recompute it at decision time. Staleness becomes impossible by construction.	The fact is deterministic and cheap (regenerate the projection, recompute the drift).	**The fact cannot be reproduced** (human/LLM verdict, paid CI, external receipt).
Snapshot-diff freshness gate	Persist the projection snapshot the claim was made against; diff against a fresh one at acceptance.	The fact is already shaped as a projection snapshot.	The fact lives outside the projection (waivers, approvals, receipts).
Certificate / version-binding	Issue each verdict bound to the fact-version it certifies.	You want freshness to travel locally with the verdict.	Multi-fact claims (reintroduces a read-set); TTL variants are unsound (wall-clock ≠ facts-changed).

The takeaway for a DCB practitioner: **don't reach for a read-set + append- condition ledger when a content hash or a rerun would do.** A deterministic anchor-rename, a doc/code drift, a regenerated registry — these are all caught trivially by hashing or rerunning, with zero standing state.

DCB survives the cost-justification test on exactly one residual class:

> **Non-rerunnable facts that participate in a concurrent acceptance.** Human approvals and waivers. LLM/agent verdicts (you can't reproduce the exact generation). Paid or non-deterministic CI. External evidence receipts. Two promotions racing on the same feature. For these, "just rerun it" is not available, and content-hashing a thing you can't regenerate doesn't help you reason about *who accepted what, against which version, in what order.* That's the causality/concurrency story only DCB tells cleanly.

The toy proof we actually ran

To stop hand-waving, we built a bounded TypeScript witness for an `OrderFulfillmentSaga` and wired a DCB freshness check on top of it. This is a **toy and a bounded witness, not a formal proof** — stated plainly so the claim never exceeds the evidence.

The scenario

A four-participant saga (ordering, payment, inventory, shipping) with forward operations, compensations, and policies. The deliberately interesting fact is a **human waiver**:

> `HumanWaiverApplied(waiver:ship-after-payment-timeout-v1)` — lets `CreateShipment` proceed after a payment-provider timeout *when an external authorization receipt exists.* It is non-rerunnable on purpose: it encodes a human authorization at a point in time.

The bounded trace (behavioral witness)

```
CreateOrder         -> success
AuthorizePayment    -> timeout   (ExternalEvidenceReceiptRecorded: payment-auth-provider-
r42)
HumanWaiver         -> waived    (permits continuation)
ReserveInventory    -> success
CreateShipment      -> failed
ReleaseInventory    -> compensated
VoidPayment         -> compensated
CancelOrder         -> compensated
DeadLetterPolicy    -> terminal-failure
```

Behavioral assertions — all **pass**:

- A run where `CreateShipment` fails after `ReserveInventory` succeeds **must not** be accepted as saga success.
- All completed compensatable steps **are** compensated.
- Removing a compensation edge **is** detected as a graph violation.

The read set — note the split mechanism

Fact	Mechanism
`SagaDeclared: OrderFulfillmentSaga`	per-dependency **hash**
`SagaStepDeclared: CreateShipment`	per-dependency **hash**
`CompensationOperationDeclared: ReleaseInventory`	per-dependency **hash**
`PolicyObserved: CompensationOrderPolicy`	per-dependency **hash**
`PropertyTestRunRecorded: run-ts-bounded-001`	per-dependency **hash**
`HumanWaiverApplied: ship-after-payment-timeout-v1`	**DCB**
`ExternalEvidenceReceiptRecorded: payment-auth-provider-r42`	**DCB**

Five deterministic facts → guarded by content hashes. Two non-rerunnable facts → guarded by DCB. This is the whole thesis made concrete in one table.

The freshness verdict

We then mutated the non-rerunnable fact: the waiver advanced from `v1` to `v2` *after* the read marker.

```
REJECTED — non-rerunnable fact HumanWaiverApplied:ship-after-payment-timeout
           changed after the read marker; DCB freshness rejects acceptance.

  stale fact   : waiver:ship-after-payment-timeout-v1
  current fact : waiver:ship-after-payment-timeout-v2
```

The behavioral witness still **passed** — the saga logic was fine. But the *acceptance* of that witness as current evidence was correctly **rejected**, because a fact it depended on (one a validator would never re-check, and one you cannot rerun) had moved. That is precisely the seam where DCB, and only DCB, adds value.

What this means if you're applying DCB for correctness

1. **Separate the two questions.** "Is it correct?" is owned by your proof / validator / test surface. "Is this accepted claim still fresh?" is owned by a DCB boundary. Don't let the boundary try to prove behavior.
2. **Name the command for what it does.** `AcceptXFreshnessClaim`, not `AcceptXCorrectnessClaim`. The name keeps the ownership honest and stops the boundary from absorbing responsibilities it can't fulfill.
3. **Triage every dependency before you put it in a boundary.** - Deterministic + cheap to recompute → **rerun-before-use** (no state). - Deterministic but expensive → **content-hash** the dependency set. - **Non-rerunnable + concurrent** → this is the one that justifies a real DCB read-set + append-condition. A boundary made entirely of rerunnable facts is over-built — it just re-implements your validators with more machinery.
4. **A read marker is a content rollup, not a clock.** We landed on a per-dependency content hash set as the marker (keeps DCB's per-fact granularity and catches change-and-revert that timestamps miss). Wall-clock TTLs are an unsound proxy for "did the facts change."
5. **Start tiny.** No event store. An append-only ledger (even JSONL) plus a single boundary over one feature is enough to prove or kill the value. We did not need projections, live read-models, or an event-store commitment to get a real rejection out of a real stale fact.

Honest boundaries (claim ≤ proof)

- The saga result is a **bounded TypeScript witness**, not a formal proof. It exercises specific traces; it does not exhaust the state space.

- Everything here is a **toy**. No production event store was built, no validators were replaced, no templates were changed.
- DCB was validated for the **freshness/concurrency** role only. We did **not** find it viable as a behavioral proof engine, as a validator replacement, or as a mandatory annotation on every spec rule.
- Open residue we did not close: coarse rollup hash vs per-dependency hashes as a default; a semantic-vs-syntactic escape hatch (content hashes over-reject cosmetic edits); and the exact relationship typing for saga orchestration in the underlying spec vocabulary.

One-paragraph summary you can paste into a message

> We tested whether Dynamic Consistency Boundaries could underwrite spec-to-code (and saga) correctness. They can't *prove* correctness — that stays with validators and tests. What DCB uniquely adds is a **freshness guard on the acceptance of a correctness claim**: it reuses the read-set + append-condition to reject a claim if a relevant fact changed after the claim read it. But for the large class of deterministic, cheaply re-derivable facts, a content hash or a rerun-before-use policy solves the same problem with far less machinery — so DCB should be reserved for the residual hard class: **non-rerunnable facts that need concurrency/causality** (human waivers, LLM/agent verdicts, paid CI, external receipts, concurrent promotions). A toy `OrderFulfillmentSaga` proved the seam end to end: five deterministic facts guarded by hashes, one human waiver guarded by DCB, and acceptance correctly **rejected** when the waiver changed — while the behavioral witness itself still passed.